

Freie Universität Berlin

Computational Sciences 25/26

PATTERN FORMATION

Project 4 Assignments Report

Group Members:

Julius Merten, Ziheng Guo, Guowei Huang, Seongmook Lim,
Weiqi Li, Soonjung Kim, Ahmed Hamed Elqamel,

Lecturers:

Sebastian Matera, Luca Donati

April 2, 2026

Abstract

This project investigates the mathematical foundations and numerical simulation of Turing instability within the Gray-Scott reaction-diffusion model to replicate biological pattern formation, specifically leopard spots and giraffe patches.

We initially implemented the explicit Euler and implicit Crank-Nicolson (CN) finite difference schemes. To guarantee mathematical reliability, we rigorously validated our numerical methods, proving that our shared spatial discretization achieves strict $\mathcal{O}(\Delta x^2)$ accuracy, while the explicit and implicit temporal integrations perfectly match their theoretical $\mathcal{O}(\Delta t)$ and $\mathcal{O}(\Delta t^2)$ convergence orders.

Despite their mathematical accuracy, baseline CPU profiling revealed major performance bottlenecks: the explicit method was strictly limited by CFL stability constraints and memory bandwidth, while the traditional sparse-matrix implicit solver faced massive memory overhead and slow direct matrix inversion times.

To overcome these computational limitations, we engineered two highly optimized architectures. First, we developed a GPU-accelerated explicit solver utilizing CuPy, which leverages massive data-level parallelism and zero-copy memory management to reduce the simulation time of 50,000 iterations to less than a second. Second, we implemented a Matrix-Free architecture for the implicit CN scheme, paired with a Conjugate Gradient (CG) iterative solver.

This matrix-free approach completely skips large sparse matrix assembly, keeping the spatial memory complexity strictly bounded to $\mathcal{O}(N^2)$ while maintaining unconditional stability through vectorized CPU operations. Automated benchmarking confirms that both optimized solvers successfully generate the target patterns while delivering significant speed and memory improvements over traditional procedural implementations.

Contents

1	Turing instability	4
1.1	Foundations	4
1.2	Turing Instability	8
2	The Grey-Scott mode	9
2.1	The Grey-Scott model	9
2.2	Numerical schemes for the reaction diffusion equation	9
2.3	Numerical stability analysis	12
3	Implementation & Architecture	16
3.1	Software Architecture, Modularity, and Quality Assurance	16
3.2	Baseline Codebase Structure and Workflow	17
3.3	Explicit Solver and Vectorization	18
3.4	Sparse Implicit Solver (Crank-Nicolson)	18
3.5	Biological Initialization and Seeding Strategies	19
4	Parallelization	20
4.1	GPU-Accelerated Explicit Reaction-Diffusion System Solver based on CuPy	20
4.1.1	Introduction and Purpose	20
4.1.2	Mathematical Principles and Parallelization Mechanisms	20
4.1.3	Code Architectural Analysis	21
4.1.4	Conclusion	22
4.2	Solving the Reaction-Diffusion System using a Matrix-Free Implicit Strategy	22
4.2.1	Introduction and Purpose	22
4.2.2	Mathematical Principles	22
4.2.3	Code Architectural Analysis	23
4.2.4	Conclusion	24
5	Results & Performance	25
5.1	Numerical Convergence and Accuracy Validation	25
5.2	Pattern Generation Analysis	26
5.3	Performance and Algorithm Benchmarking Before Optimization	26
5.4	Performance and Algorithm Benchmarking Following Parallelization Optimization	30

6 Conclusion	33
References	34

1 Turing instability

Alan Turing made some groundbreaking discovery by analyzing the reaction diffusion equation. He discovered that under certain instability conditions for the reaction diffusion equation, patterns form. These patterns remind of animal fur patterns, e.g. from a giraffe or leopard. For this reason Turings discoveries were interesting in the field of evolution theory and so on.

1.1 Foundations

For first we want to define our notions rigorously. We are looking at the reaction diffusion equation:

$$\begin{aligned}u_t &= \delta_1 \Delta u + f(u, v), \\v_t &= \delta_2 \Delta v + g(u, v).\end{aligned}$$

Where $u, v : [0, T] \times [0, 1]^2 \rightarrow \mathbb{R}$, $f, g : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $\delta_1, \delta_2 > 0$. And we are considering periodic boundary conditions. Indices for a function representing a variable resemble the partial derivative to corresponding variable, e.g. u_t resembles the partial of u with respect to time variable t .

If we were now asked to visualize a solution of this reaction diffusion PDE, we could just fix our time parameter and plot a unit square with a certain coloring, where each color on a spectrum represents a certain value attained at this point. For a different time point, we might have a different colored unit square. So one could think about the (visualized) solution as an colored unit square paper stack, where the stack height corresponds to a certain time point and each colored square sheet has infinitesimal thickness. It might help to keep this picture in mind. From now on we will start with the stability analysis of this PDE itself.

We will look at a homogenous solution (u_0, v_0) and perturb it a little. Whenever a homogenous solution exists (such a solution corresponds to a one colored plot), we have that $f(u_0, v_0) = g(u_0, v_0) = 0$ has to hold, since the derivatives of u_0 and v_0 simply vanish, when we plug this into our PDE. This has the neat side effect of making a linearization of the so called reaction terms f and g very practical. We have that:

$$f(u, v) \approx f(u_0, v_0) + Df(u_0, v_0)(u - u_0, v - v_0)^t = f_u(u_0, v_0)(u - u_0) + f_v(u_0, v_0)(v - v_0).$$

And the same holds for the reaction term g , expanded around (u_0, v_0) . For convenience we will just use the shorthand notation $f_u := f_u(u_0, v_0)$ and so on, since we will not be dealing with partials evaluated at different points for the reaction terms. If we plug in our linearization approach into our PDE, we get:

$$\begin{aligned}u_t &\approx \delta_1 \Delta u + f_u(u - u_0) + f_v(v - v_0), \\v_t &\approx \delta_2 \Delta v + g_u(u - u_0) + g_v(v - v_0).\end{aligned}$$

From now on let us consider an exact PDE of this form in $U = u - u_0$ and $V = v - v_0$:

$$\begin{aligned}U_t &= \delta_1 \Delta U + f_u U + f_v V, \\V_t &= \delta_2 \Delta V + g_u U + g_v V.\end{aligned}$$

This reaction diffusion equation should follow periodic boundary conditions as well, leading to our next approach: Fourier expansion.

Since the maps U and V are said to be periodic, we will assume them to have a corresponding Fourier series expansion:

$$U(t, x, y) = \sum_{k=(k_x, k_y) \in \mathbb{Z}^2} \hat{u}_k(t) e^{i(k_x x + k_y y)}, \quad V(t, x, y) = \sum_{k=(k_x, k_y) \in \mathbb{Z}^2} \hat{v}_k(t) e^{i(k_x x + k_y y)}.$$

Analysing now one term, we get with our PDE that:

$$\begin{aligned} \hat{u}'_k e^{i(k_x x + k_y y)} &= \delta_1 \hat{u}_k (k_x^2 + k_y^2) e^{i(k_x x + k_y y)} + f_u \hat{u}_k e^{i(k_x x + k_y y)} + f_v \hat{v}_k e^{i(k_x x + k_y y)} \\ \Leftrightarrow \hat{u}'_k &= -\delta_1 \hat{u}_k (k_x^2 + k_y^2) + f_u \hat{u}_k + f_v \hat{v}_k. \end{aligned}$$

And the same holds for $\hat{v}_k$:

$$\hat{v}'_k = -\delta_2 \hat{v}_k (k_x^2 + k_y^2) + g_u \hat{u}_k + g_v \hat{v}_k.$$

Combining these coefficient ODEs, we get one ODE matrix form with $X_k = (\hat{u}_k, \hat{v}_k)^t$:

$$(X_k)_t = \begin{pmatrix} -\delta_1(k_x^2 + k_y^2) + f_u & f_v \\ g_u & -\delta_2(k_x^2 + k_y^2) + g_v \end{pmatrix} X_k.$$

We will try to solve this with the typical eigenvalue, eigenvector approach. And for A_k we get the following characteristic polynomial:

$$\begin{aligned} &\begin{vmatrix} -\delta_1(k_x^2 + k_y^2) + f_u - \lambda_k & f_v \\ g_u & -\delta_2(k_x^2 + k_y^2) + g_v - \lambda_k \end{vmatrix} = 0 \\ \Leftrightarrow &(-\delta_1(k_x^2 + k_y^2) + f_u - \lambda_k)(-\delta_2(k_x^2 + k_y^2) + g_v - \lambda_k) - f_v g_u = 0 \\ \Rightarrow &[\delta_1 \delta_2 (k_x^2 + k_y^2)^2 - \delta_1(k_x^2 + k_y^2) g_v - \delta_2(k_x^2 + k_y^2) f_u + f_u g_v - f_v g_u] \\ &+ [\delta_1(k_x^2 + k_y^2) + \delta_2(k_x^2 + k_y^2) - f_u - g_v] \lambda_k + \lambda_k^2 = 0. \end{aligned}$$

Let us now look at the discriminant D_k :

$$\begin{aligned} D_k &= [\delta_1(k_x^2 + k_y^2) + \delta_2(k_x^2 + k_y^2) - f_u - g_v]^2 \\ &\quad - 4[\delta_1 \delta_2 (k_x^2 + k_y^2)^2 - \delta_1(k_x^2 + k_y^2) g_v - \delta_2(k_x^2 + k_y^2) f_u + f_u g_v - f_v g_u]. \end{aligned}$$

We have $f_u + g_v = \text{tr}(J :=)$ with $\phi(u, v) := (f(u, v), g(u, v))^t$ and $f_u g_v - f_v g_u = \det(J)$.

We then get:

$$\begin{aligned} D_k &= (\delta_1 + \delta_2)^2 (k_x^2 + k_y^2)^2 - 2(\delta_1 + \delta_2)(k_x^2 + k_y^2) \text{tr}(J) + \text{tr}^2(J) \\ &\quad - 4[\delta_1 \delta_2 (k_x^2 + k_y^2)^2 - \delta_1(k_x^2 + k_y^2) g_v - \delta_2(k_x^2 + k_y^2) f_u + \det(J)]. \end{aligned}$$

We can now simplify this equality to:

$$\begin{aligned} D_k &= [(\delta_1 - \delta_2)^2 (k_x^2 + k_y^2)^2 + 2(g_v - f_u)(\delta_1 - \delta_2)(k_x^2 + k_y^2) + (g_v - f_u)^2] + 4f_v g_u \\ &= [(\delta_1 - \delta_2)(k_x^2 + k_y^2) + (g_v - f_u)]^2 + 4f_v g_u. \end{aligned}$$

We now get the Eigenvalues:

$$\lambda_k^\pm = \frac{-(k_x^2 + k_y^2)(\delta_1 + \delta_2) + \text{tr}(J) \pm \sqrt{[(\delta_1 - \delta_2)(k_x^2 + k_y^2) + (g_v - f_u)]^2 + 4f_v g_u}}{2}.$$

And we get corresponding eigenvectors ρ_k^+ and ρ_k^- . We can now write our solution for the coefficient ODE as:

$$X_k = \rho_k^+ e^{\lambda_k^+ t} + \rho_k^- e^{\lambda_k^- t} = (\hat{u}_k, \hat{v}_k)^t.$$

We have that

$$X := (U, V)^t = \sum_{k \in \mathbb{Z}^2} (\hat{u}_k, \hat{v}_k)^t e^{i(k_x x + k_y y)} = \sum_{k \in \mathbb{Z}^2} (\rho_k^+ e^{\lambda_k^+ t} + \rho_k^- e^{\lambda_k^- t}) e^{i(k_x x + k_y y)}.$$

In fact we could have complex eigenvalues. Let us define $K := k_x^2 + k_y^2$ then get for $D_k \leq 0$ that:

$$\text{Re}(\lambda_k^\pm) = \frac{-K(\delta_1 + \delta_2) + \text{tr}(J)}{2} \leq \frac{\text{tr}(J)}{2}.$$

But if $D_k > 0$ meaning we do not get complex eigenvalues. We then have:

$$\begin{aligned} \lambda_k^\pm &\leq \lambda_k^+ = \frac{-K(\delta_1 + \delta_2) + \text{tr}(J)}{2} + \frac{\sqrt{[(\delta_1 - \delta_2)K + (g_v - f_u)]^2 + 4f_v g_u}}{2} \\ &\leq \frac{|(\delta_1 - \delta_2)K + (g_v - f_u)| - K(\delta_1 + \delta_2)}{2} + \frac{\text{tr}(J)}{2} + \sqrt{|f_v g_u|} \\ &\leq \frac{K(|\delta_1 - \delta_2| - (\delta_1 + \delta_2)) + |g_v - f_u|}{2} + \frac{\text{tr}(J)}{2} + \sqrt{|f_v g_u|}. \end{aligned}$$

We have that $|\delta_1 - \delta_2| - (\delta_1 + \delta_2) = -2 \min(\delta_1, \delta_2) < 0$ Giving us:

$$\lambda_k^\pm \leq \frac{|g_v - f_u| + \text{tr}(J)}{2} + \sqrt{|f_v g_u|}.$$

Therefore our eigenvalues are bounded in their real part from above.

Whenever we want to discover some trend in our long term behavior of our PDE solution, we are interested in the real parts. The imaginary parts just correspond to some oscillating dynamic. But since the real parts are bounded from above and $\text{Re}(\lambda_k^\pm) \rightarrow -\infty$ as $K \rightarrow \infty$, we can find a maximum value $\Lambda := \max\{\text{Re}(\lambda_k^+), \text{Re}(\lambda_k^-) : k \in \mathbb{Z}^2\}$.

This mode dominates our series as $t \rightarrow \infty$ and describes our pattern growth. But the shape of our pattern is described by K' , where $\text{Re}(\lambda_{k: k_x^2 + k_y^2 = K'}^+) = \Lambda$. Big K' 's describe a very "detailed" pattern. Small K' 's describe a pattern with comparable bigger spots or stripes, respectively a rougher structure. In fact for $K = 0$, we have an additional special case: Even though our corresponding mode might not vanish over time, this mode will not impact the special pattern, since it is constant in space.

However, we can also see directly that the long term behavior of our solution will tend to attain a homogenous state again, i.e. converging to 0, whenever we are dealing with only negative eigenvalues. In such a case, no patterns would form. Nevertheless, we will always have negative eigenvalues for large enough K , as our inequality shows. This will happen at last when:

$$K \geq \frac{|g_v - f_u| + \text{tr}(J) + 2\sqrt{|f_v g_u|}}{2 \min(\delta_1, \delta_2)}.$$

So in fact the set of non-negative eigenvalues is finite.

As one might already guess, the interesting part is the case where our PDE is instable for at least one mode. In such a case patterns can form over time. However, our linear PDE will get worse as an approximation to our original reaction diffusion PDE, since our special error will become bigger and bigger over time. In fact one can see that it grows exponentially. But the vanishing modes also decay exponentially.

Let us now state some final requirements for the pattern to be forming. We have the following equation for our eigenvalues or modes:

$$[\delta_1\delta_2K^2 - \delta_1Kg_v - \delta_2Kf_u + f_ug_v - f_vg_u] + [\delta_1K + \delta_2K - f_u - g_v]\lambda_k + \lambda_k^2 = 0.$$

Where $K := k_x^2 + k_y^2$. Our requirement for patterns to form is that we have at least one positive mode or eigenvalue. Otherwise our PDE would be so stable that it will re-attain the homogenous state over time. Our equation from above has the following solution:

$$\lambda_k = \frac{-K(\delta_1 + \delta_2) + \text{tr}(J)}{2} \pm \frac{\sqrt{D_k}}{2}.$$

With $D_k = [(\delta_1 - \delta_2)K + (g_v - f_u)]^2 + 4f_vg_u$. We have the case distinction, where $D_k < 0$, i.e. we get complex eigenvalues or $D_k \geq 0$. Since the real part of our solution really matters for its growth over time, we just care whether the real part is larger or smaller than zero. In the case $D_k < 0$ we have

$$\text{Re}(\lambda_k) = \frac{-K(\delta_1 + \delta_2) + \text{tr}(J)}{2}.$$

In case that $D_k \geq 0$, we have that

$$\text{Re}(\lambda_k) = \lambda_k = \frac{-K(\delta_1 + \delta_2) + \text{tr}(J)}{2} \pm \frac{\sqrt{D_k}}{2}.$$

We then get for $\text{Re}(\lambda_k) > 0$ the requirement on $K \in \mathbb{R}_{\geq 0}$ that:

$$D_k < 0 \Rightarrow \frac{\text{tr}(J)}{(\delta_1 + \delta_2)} > K$$

$$D_k \geq 0 \Rightarrow \pm\sqrt{D_k} > K(\delta_1 + \delta_2) - \text{tr}(J)$$

In both cases a second case distinction comes into play: the sign of $\text{tr}(J)$. When we have $\text{tr}(J) < 0$ we will have guaranteed stability in the first case. For the second case, we have $K(\delta_1 + \delta_2) - \text{tr}(J) > 0$ and therefore $\sqrt{D_k} > K(\delta_1 + \delta_2) - \text{tr}(J)$ has to hold, the second eigenvalue ("with minus") will always be negative. We now have that:

$$D_k > (K(\delta_1 + \delta_2) - \text{tr}(J))^2$$

has to hold. This gives us the equivalent requirement:

$$\det(J) - K(\delta_2f_u + \delta_1g_v) + \delta_1\delta_2K^2 < 0.$$

In the latter case, we will check for the largest K , satisfying our condition.

1.2 Turing Instability

In chapter 1.1 we discussed the basic setting to analyse our problem. We talked about steady states and linearization to do so. We have also seen that even though our system is linear, the whole analysis of this simplified setting becomes very messy.

But there is a brilliant concept out of this quagmire, i.e. separating the diffusion terms and the reaction terms. In theory one often considers the linearised reaction diffusion equation with stable reaction terms. This means that the PDE is stable just in the reaction terms, i.e. for $\delta_1 = \delta_2 = 0$. For the eigenvalues in this case, we have guaranteed stability, if $\text{tr}(J) < 0$ and $\det(J) > 0$, since they have the form:

$$\lambda^\pm = \frac{\text{tr}(J) \pm \sqrt{(\text{tr}(J))^2 - 4\det(J)}}{2} \leq \lambda^+ = \frac{\text{tr}(J) + \sqrt{(\text{tr}(J))^2 - 4\det(J)}}{2}.$$

Whenever we have instability despite a stable reaction setting in our equation, we talk about Turing mechanism or Turing instability. Excluding these cases makes our life more easy and we can get more precise statements.

We can now restrict our discussion from before to the cases $\text{tr}(J) < 0$, where Turing instability can just appear if $D_k \geq 0$. We then end up with the requirement

$$\det(J) - K(\delta_2 f_u + \delta_1 g_v) + \delta_1 \delta_2 K^2 < 0$$

on the modes to be instable we can do some rearrangements and see that:

$$\delta_1 \delta_2 \left(K - \frac{\delta_2 f_u + \delta_1 g_v}{2\delta_1 \delta_2} \right)^2 + \det(J) - \frac{(\delta_2 f_u + \delta_1 g_v)^2}{4\delta_1 \delta_2} < 0.$$

Meaning modes can just attain instability if $\det(J) < \frac{(\delta_2 f_u + \delta_1 g_v)^2}{4\delta_1 \delta_2}$. That's a necessary criterion and it is delicate since we assume $\det(J) > 0$. Why is it just necessary? Because our

$$K \in \{K = k_x^2 + k_y^2 : (k_x, k_y) \in \mathbb{Z}^2\}$$

i.e. they lie in a discrete set. We also have to make sure that a minimum can be attained for our quadratic in K , meaning $\delta_2 f_u + \delta_1 g_v > 0$, to lie in our K domain. This gives us the standard Turing instability requirements:

1. We consider just settings with stable reaction dynamic, i.e.

$$\text{tr}(J) < 0 \text{ and } \det J > 0.$$

2. To have instability (by diffusion or Turing instability) for such a setting, we need additionally

$$\left[\delta_2 f_u + \delta_1 g_v > 0 \text{ and } \det J < \frac{(\delta_2 f_u + \delta_1 g_v)^2}{4\delta_1 \delta_2} \right] \Leftrightarrow \delta_2 f_u + \delta_1 g_v > 2\sqrt{\delta_1 \delta_2 \det J},$$

which is a necessary criterion.

2 The Grey-Scott mode

2.1 The Grey-Scott model

In the Grey-Scott model we have the reaction terms f and g explicitly given as:

$$\begin{aligned} f(u, v) &= -uv^2 + \alpha(1 - u) \\ g(u, v) &= uv^2 - (\alpha + \beta)v. \end{aligned}$$

With $\alpha, \beta \in \mathbb{R}_{>0}$. Calculating the Jacobian of $\phi = (f, g)^t$ leads to:

$$J = \begin{pmatrix} -v^2 - \alpha & -2uv \\ v^2 & 2uv - (\alpha + \beta) \end{pmatrix}.$$

However, we are just interested in points (u, v) , where $f(u, v) = g(u, v) = 0$. We will now do a Turing analysis following the conditions of the previous chapter. First we check on the base assumptions, i.e. whether:

$$\det(J) = -(v^2 + \alpha)(2uv - (\alpha + \beta)) + 2uv^3 > 0$$

and

$$\text{tr}(J) = 2uv - (\alpha + \beta) - (v^2 + \alpha) < 0.$$

Using $f(u, v) = g(u, v) = 0$, we can conclude that $uv = \alpha + \beta$ for $v \neq 0$. We then get:

$$\begin{aligned} \det(J) &= (v^2 - \alpha)(\alpha + \beta) > 0, \\ \text{tr}(J) &= \beta - v^2 < 0. \end{aligned}$$

Giving us the short requirements $v^2 > \max(\alpha, \beta)$.

If we consider the so called trivial equilibrium point $(u, v) = (1, 0)$, i.e. the only point with $v = 0$ leading to an equilibrium, we get that:

$$\det(J) = \alpha(\alpha + \beta), \quad \text{tr}(J) = -2\alpha - \beta.$$

Fulfilling our base requirements. But we also get that $-\alpha\delta_2 - (\alpha + \beta)\delta_1 < 0$ meaning that our necessary criterion fails and we will never have a positive mode at this trivial point of equilibrium. So the case where $v^2 > \max(\alpha, \beta)$ remains the only interesting one for Turing instability. We have that $f_u = -(v^2 + \alpha) < 0$ and $g_v = 2uv - (\alpha + \beta) = \alpha + \beta$. And thus when we check for $\delta_2 f_u + \delta_1 g_v > 2\sqrt{\delta_1 \delta_2 \det(J)}$, where $\det(J) = (\alpha + \beta)(v^2 - \alpha)$, we get that:

$$-\delta_2(v^2 + \alpha) + \delta_1(\alpha + \beta) > 2\sqrt{\delta_1 \delta_2 (\alpha + \beta)(v^2 - \alpha)}.$$

2.2 Numerical schemes for the reaction diffusion equation

So we consider the following problem: Let $\Omega := [0, 1]^2$ and $T \in \mathbb{R}_{>0}$ and $u, v : [0, T] \times \Omega \rightarrow \mathbb{R}$.

We now consider the PDE system:

$$\begin{aligned} u_t &= \delta_1 \Delta u + f(u, v) \\ v_t &= \delta_2 \Delta v + g(u, v) \end{aligned}$$

where $\delta_1, \delta_2 \in \mathbb{R}_{>0}$ and $f, g \in C^0(\mathbb{R}^2; \mathbb{R})$.

Note that $\Delta u = u_{xx} + u_{yy}$ and $\Delta v = v_{xx} + v_{yy}$. For second order derivatives we will use central finite difference approximation. But we need to be a little more careful for this case, since we are dealing with an subset of $\mathbb{R}^2$ domain in space. For this reason, we will first have to establish a grid for further discretisation of the problem. We will choose an equidistant grid on our domain, i.e. a grid of the form:

$$\{p_{i,j} := (i/n, j/n) : i, j \in [n] = \{0, \dots, n\}\}.$$

Now any value of either u or v shall be approximated at point $p_{i,j}$ by $u_{i,j}$ resp. $v_{i,j}$. Since we have now established our notation, we may use the CFD approximation for our functions:

$$\begin{aligned} \Delta u|_{p_{i,j}} &\approx n^2(u_{i+1,j} - 2u_{i,j} + u_{i-1,j} + u_{i,j+1} - 2u_{i,j} + u_{i,j-1}), \\ \Delta v|_{p_{i,j}} &\approx n^2(v_{i+1,j} - 2v_{i,j} + v_{i-1,j} + v_{i,j+1} - 2v_{i,j} + v_{i,j-1}). \end{aligned}$$

We used the same spatial step size for both approximations in the first and second (spatial) argument of u and v . Therefore we end up with the following semi-discrete scheme in time:

$$\begin{aligned} u_t|_{p_{i,j}} &\approx \delta_1 n^2(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}) + f(u_{i,j}, v_{i,j}), \\ v_t|_{p_{i,j}} &\approx \delta_2 n^2(v_{i+1,j} + v_{i-1,j} + v_{i,j+1} + v_{i,j-1} - 4v_{i,j}) + g(u_{i,j}, v_{i,j}). \end{aligned}$$

So far we just ignored the dependence on the time variable. But in fact, we have to discretise in time as well. We can easily do this since we just developed our semi-discrete scheme. The easiest approach might be an explicit Euler approach or (to touch an implicit method as well) an implicit Euler. Later on, we will also look at the implicit midpoint rule, the "swiss-army-knife" of the numerical integrators. In either way, we develop some approximation of the first derivative in time. Let us choose an equidistant 1D grid $0 = t_0 < t_1 < \dots < t_m = T$ where $t_k := \frac{k}{m}T$ with $k \in [m]$. Approximations on the k -th time step will be denoted by the exponent k .

We then have for the explicit Euler:

$$\begin{aligned} u_t|_{p_{i,j}} &\approx \frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m} = \delta_1 n^2(u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) + f(u_{i,j}^k, v_{i,j}^k), \\ v_t|_{p_{i,j}} &\approx \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m} = \delta_2 n^2(v_{i+1,j}^k + v_{i-1,j}^k + v_{i,j+1}^k + v_{i,j-1}^k - 4v_{i,j}^k) + g(u_{i,j}^k, v_{i,j}^k). \end{aligned}$$

And for implicit Euler we get:

$$\begin{aligned} \frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m} &= \delta_1 n^2(u_{i+1,j}^{k+1} + u_{i-1,j}^{k+1} + u_{i,j+1}^{k+1} + u_{i,j-1}^{k+1} - 4u_{i,j}^{k+1}) + f(u_{i,j}^{k+1}, v_{i,j}^{k+1}), \\ \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m} &= \delta_2 n^2(v_{i+1,j}^{k+1} + v_{i-1,j}^{k+1} + v_{i,j+1}^{k+1} + v_{i,j-1}^{k+1} - 4v_{i,j}^{k+1}) + g(u_{i,j}^{k+1}, v_{i,j}^{k+1}). \end{aligned}$$

We now want to solve for the consecutive time step. For this purpose, we establish the vector:

$$\begin{aligned} u^k &:= (u_{0,0}, \dots, u_{0,n}, u_{1,0}, \dots, u_{1,n}, \dots, u_{n,0}, \dots, u_{n,n})^t \\ v^k &:= (v_{0,0}, \dots, v_{0,n}, v_{1,0}, \dots, v_{1,n}, \dots, v_{n,0}, \dots, v_{n,n})^t. \end{aligned}$$

Rearranging our equation gives us for explicit Euler the following:

$$\begin{aligned} u_{i,j}^{k+1} &= \frac{\delta_1 n^2 T}{m} (u_{i+1,j}^k + u_{i-1,j}^k + u_{i,j+1}^k + u_{i,j-1}^k - 4u_{i,j}^k) + u_{i,j}^k + \frac{T}{m} f(u_{i,j}^k, v_{i,j}^k), \\ v_{i,j}^{k+1} &= \frac{\delta_2 n^2 T}{m} (v_{i+1,j}^k + v_{i-1,j}^k + v_{i,j+1}^k + v_{i,j-1}^k - 4v_{i,j}^k) + v_{i,j}^k + \frac{T}{m} g(u_{i,j}^k, v_{i,j}^k). \end{aligned}$$

In vectorial form we then end up with:

$$\begin{aligned} u^{k+1} &= \frac{\delta_1 n^2 T}{m} (A_{i+1,j} + A_{i-1,j} + A_{i,j+1} + A_{i,j-1}) u^k + \left(1 - 4 \frac{\delta_1 n^2 T}{m}\right) E u^k + \frac{T}{m} F(u^k, v^k), \\ v^{k+1} &= \frac{\delta_2 n^2 T}{m} (A_{i+1,j} + A_{i-1,j} + A_{i,j+1} + A_{i,j-1}) v^k + \left(1 - 4 \frac{\delta_2 n^2 T}{m}\right) E v^k + \frac{T}{m} G(u^k, v^k). \end{aligned}$$

Where E is the identity matrix and since we are dealing with periodic boundary conditions, the matrices $A_{i\pm 1,j}$ and $A_{i,j\pm 1}$ are in such a way, that the special indices of either the first or second variable are shifted by ± 1 . The last summands $F(u^k, v^k)$ and $G(u^k, v^k)$ refer to the vectors:

$$\begin{aligned} F(u^k, v^k) &= (f(u_{0,0}^k, v_{0,0}^k), \dots, f(u_{n,n}^k, v_{n,n}^k))^t, \\ G(u^k, v^k) &= (g(u_{0,0}^k, v_{0,0}^k), \dots, g(u_{n,n}^k, v_{n,n}^k))^t. \end{aligned}$$

Let us take a closer look at the "shifting" matrices $A_{i\pm 1,j}$ and $A_{i,j\pm 1}$. We have that e.g.

$$A_{i+1,j} u^k = (u_{1,0}, \dots, u_{1,n}, \dots, u_{n,0}, \dots, u_{n,n}, u_{0,0}, \dots, u_{0,n})^t.$$

Meaning that:

$$A_{i+1,j} = \begin{pmatrix} 0_{n+1} & 0_{n+1} & \cdots & 0_{n+1} & id_{n+1} \\ id_{n+1} & 0_{n+1} & \cdots & 0_{n+1} & 0_{n+1} \\ 0_{n+1} & id_{n+1} & \cdots & 0_{n+1} & 0_{n+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0_{n+1} & 0_{n+1} & \cdots & id_{n+1} & 0_{n+1} \end{pmatrix} \in \mathbb{R}^{(n+1)^2 \times (n+1)^2}$$

where 0_{n+1} denotes the matrix in $\mathbb{R}^{(n+1) \times (n+1)}$ and id_{n+1} the identity matrix in $\mathbb{R}^{(n+1) \times (n+1)}$. So whenever we are doing a matrix vector multiplication with a vector $x := (x_0, \dots, x_n)^t \in \mathbb{R}^{(n+1)^2}$, where $x_i \in \mathbb{R}^{n+1}$, we have a component wise (regarding the matrix components) matrix vector multiplication in $\mathbb{R}^{n+1}$. So for $A_{i-1,j}$ we obtain the following matrix:

$$A_{i-1,j} = \begin{pmatrix} 0_{n+1} & id_{n+1} & 0_{n+1} & \cdots & 0_{n+1} \\ 0_{n+1} & 0_{n+1} & id_{n+1} & \cdots & 0_{n+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0_{n+1} & 0_{n+1} & 0_{n+1} & \cdots & id_{n+1} \\ id_{n+1} & 0_{n+1} & 0_{n+1} & \cdots & 0_{n+1} \end{pmatrix}.$$

For the matrices shifting the second time variable are something different. Here we have e.g.

$$A_{i,j+1}u^k = (u_{0,1}, \dots, u_{0,n}, u_{0,0}, u_{1,1}, \dots, u_{1,n}, u_{1,0}, \dots, u_{n,1}, \dots, u_{n,n}, u_{n,0})^t.$$

Meaning that a vector, say $x := (x_0, \dots, x_n)^t \in \mathbb{R}^{(n+1)^2}$, will be mapped by $A_{i,j+1}$ like:

$$x \mapsto (\Pi x_0, \dots, \Pi x_n)^t$$

where $\Pi : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$ reorders the entries in such a way that $e_1 \mapsto e_2, \dots, e_n \mapsto e_{n+1}, e_{n+1} \mapsto e_1$ i.e. the permutation matrix regarding the permutation $(1 \ 2 \ \dots \ n+1)$ in cycle notation.

For the matrix $A_{i,j-1}$ we get a similar result with the additional property, that the corresponding cycle is invers our previous one, resulting in $x \mapsto (\Pi^t x_0, \dots, \Pi^t x_n)^t$. We therefore have:

$$A_{i,j+1} = \begin{pmatrix} \Pi & \cdots & 0_{n+1} \\ \vdots & \ddots & \vdots \\ 0_{n+1} & \cdots & \Pi \end{pmatrix}, \quad A_{i,j-1} = \begin{pmatrix} \Pi^t & \cdots & 0_{n+1} \\ \vdots & \ddots & \vdots \\ 0_{n+1} & \cdots & \Pi^t \end{pmatrix}.$$

For the implicit Euler scheme we then get:

$$\begin{aligned} u_{i,j}^{k+1} - \frac{\delta_1 n^2 T}{m} (u_{i+1,j}^{k+1} + u_{i-1,j}^{k+1} + u_{i,j+1}^{k+1} + u_{i,j-1}^{k+1} - 4u_{i,j}^{k+1}) &= u_{i,j}^k + \frac{T}{m} f(u_{i,j}^{k+1}, v_{i,j}^{k+1}), \\ v_{i,j}^{k+1} - \frac{\delta_2 n^2 T}{m} (v_{i+1,j}^{k+1} + v_{i-1,j}^{k+1} + v_{i,j+1}^{k+1} + v_{i,j-1}^{k+1} - 4v_{i,j}^{k+1}) &= v_{i,j}^k + \frac{T}{m} g(u_{i,j}^{k+1}, v_{i,j}^{k+1}). \end{aligned}$$

Or in the matrix form we get:

$$\begin{aligned} \left((1 + 4 \frac{\delta_1 n^2 T}{m}) E - \frac{\delta_1 n^2 T}{m} (A_{i+1,j} + A_{i-1,j} + A_{i,j+1} + A_{i,j-1}) \right) u^{k+1} - \frac{T}{m} F(u^{k+1}, v^{k+1}) &= u^k, \\ \left((1 + 4 \frac{\delta_2 n^2 T}{m}) E - \frac{\delta_2 n^2 T}{m} (A_{i+1,j} + A_{i-1,j} + A_{i,j+1} + A_{i,j-1}) \right) v^{k+1} - \frac{T}{m} G(u^{k+1}, v^{k+1}) &= v^k. \end{aligned}$$

With a similar setting as for the explicit Euler method. This system needs some solver for the consecutive time step, since it is just implicitly given. Last but not least we will take a look at the implicit midpoint rule, aka Crank-Nicolson method. This method is essentially just the average of both methods, the implicit and explicit Euler. We therefore get:

$$\begin{aligned} \frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m} &= \frac{1}{2} [\delta_1 n^2 (u_{i+1,j}^{k+1} + \dots - 4u_{i,j}^{k+1}) + f(u_{i,j}^{k+1}, v_{i,j}^{k+1}) \\ &\quad + \delta_1 n^2 (u_{i+1,j}^k + \dots - 4u_{i,j}^k) + f(u_{i,j}^k, v_{i,j}^k)], \\ \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m} &= \frac{1}{2} [\delta_2 n^2 (v_{i+1,j}^{k+1} + \dots - 4v_{i,j}^{k+1}) + g(u_{i,j}^{k+1}, v_{i,j}^{k+1}) \\ &\quad + \delta_2 n^2 (v_{i+1,j}^k + \dots - 4v_{i,j}^k) + g(u_{i,j}^k, v_{i,j}^k)]. \end{aligned}$$

2.3 Numerical stability analysis

For the stability analysis of our scheme we will start with the semi-discrete scheme in time and do a Fourier analysis similarly to the introduction part in chapter 1. The resulting requirement on the time parameter can be

then used to analyse a stable step size for our problem.

Our semi discrete scheme reads as follows:

$$\begin{aligned}(u_{i,j})_t &= \delta_1 n^2 (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}) + f(u_{i,j}, v_{i,j}), \\ (v_{i,j})_t &= \delta_2 n^2 (v_{i+1,j} + v_{i-1,j} + v_{i,j+1} + v_{i,j-1} - 4v_{i,j}) + g(u_{i,j}, v_{i,j}).\end{aligned}$$

Similarly to chapter 1 we want to linearise our problem now, meaning we will replace:

$$\begin{aligned}f(u_{i,j}, v_{i,j}) &\mapsto f_u u_{i,j} + f_v v_{i,j} \\ g(u_{i,j}, v_{i,j}) &\mapsto g_u u_{i,j} + g_v v_{i,j}.\end{aligned}$$

Around an equilibrium point (u_0, v_0) . We will assume w.l.o.g. that $(u_0, v_0) = 0$ (see chapter 1). We can now use the Fourier approach and get for one mode that:

$$u_{i,j} = \hat{u}(t) e^{i(k_x x_i + k_y y_j)}, \quad v_{i,j} = \hat{v}(t) e^{i(k_x x_i + k_y y_j)}.$$

Plugging this into our equation yields to:

$$\begin{aligned}\hat{u}' &= \delta_1 n^2 \hat{u} (e^{i \frac{k_x}{n}} + e^{-i \frac{k_x}{n}} + e^{i \frac{k_y}{n}} + e^{-i \frac{k_y}{n}} - 4) + f_u \hat{u} + f_v \hat{v}, \\ \hat{v}' &= \delta_2 n^2 \hat{v} (e^{i \frac{k_x}{n}} + e^{-i \frac{k_x}{n}} + e^{i \frac{k_y}{n}} + e^{-i \frac{k_y}{n}} - 4) + g_u \hat{u} + g_v \hat{v}.\end{aligned}$$

We have that $e^{it} + e^{-it} = 2 \cos(t)$ and $\cos(t) - 1 = -2 \sin^2(\frac{t}{2})$. Leading to:

$$\begin{aligned}\hat{u}' &= -4\delta_1 n^2 \hat{u} \left(\sin^2\left(\frac{k_x}{2n}\right) + \sin^2\left(\frac{k_y}{2n}\right) \right) + f_u \hat{u} + f_v \hat{v}, \\ \hat{v}' &= -4\delta_2 n^2 \hat{v} \left(\sin^2\left(\frac{k_x}{2n}\right) + \sin^2\left(\frac{k_y}{2n}\right) \right) + g_u \hat{u} + g_v \hat{v}.\end{aligned}$$

We define $\xi_n := \sin^2(\frac{k_x}{2n}) + \sin^2(\frac{k_y}{2n})$.

Setting $X = (\hat{u}, \hat{v})^t$ we get the matrix ODE system:

$$X' = \begin{pmatrix} -4\delta_1 n^2 \xi_n + f_u & f_v \\ g_u & -4\delta_2 n^2 \xi_n + g_v \end{pmatrix} X.$$

Substituting $\kappa = 4n^2 \xi_n$ yields to the same eigenvalue problem as in the continuous case. For our numeric model we want to assume the Turing requirements, i.e. $\text{tr}(J) < 0$ and $\det(J) > 0$. To obtain instable modes, we require then

$$\kappa_{max} = 8n^2 > \delta_2 f_u + \delta_1 g_v > 2\sqrt{\delta_1 \delta_2 \det(J)}$$

by our explanations in chapter 2.

We will now finish the introduction part of this project by doing the von Neumann analysis on the fully discrete schemes. We know so far a lot about the eigenvalues of the semi discrete scheme. When we now discretize in time, we first go from:

$$(u_{i,j})_t \mapsto \frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m}, \quad (v_{i,j})_t \mapsto \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m}$$

for a time step $\frac{T}{m}$. The RHS of our linearized PDE then reads as follows:

$$\begin{aligned}\mathcal{U} &= \delta_1 n^2 (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}) + f_u u_{i,j} + f_v v_{i,j}, \\ \mathcal{V} &= \delta_2 n^2 (v_{i+1,j} + v_{i-1,j} + v_{i,j+1} + v_{i,j-1} - 4v_{i,j}) + g_u u_{i,j} + g_v v_{i,j}.\end{aligned}$$

We now get for the explicit Euler scheme:

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m} = \mathcal{U}^k, \quad \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m} = \mathcal{V}^k.$$

Where $\mathcal{U}^k, \mathcal{V}^k$ is the approximated time t_k for the RHS. We now make the von Neumann approach with $u_{i,j}^k = \hat{u}^k e^{i(k_x x_i + k_y y_j)}$, similar to the semi discrete Fourier analysis. And get:

$$\begin{aligned}\mathcal{U}^k e^{-i(k_x x_i + k_y y_j)} &= -4\delta_1 n^2 \xi_n \hat{u}^k + f_u \hat{u}^k + f_v \hat{v}^k, \\ \mathcal{V}^k e^{-i(k_x x_i + k_y y_j)} &= -4\delta_2 n^2 \xi_n \hat{v}^k + g_u \hat{u}^k + g_v \hat{v}^k.\end{aligned}$$

This holds for all k . We can therefore use this formula for the implicit Euler scheme as well. Solving our equations for the consecutive time step gives us:

$$\begin{aligned}\hat{u}^{k+1} &= -4\delta_1 n^2 \frac{T}{m} \xi_n \hat{u}^k + f_u \frac{T}{m} \hat{u}^k + f_v \frac{T}{m} \hat{v}^k + \hat{u}^k, \\ \hat{v}^{k+1} &= -4\delta_2 n^2 \frac{T}{m} \xi_n \hat{v}^k + g_u \frac{T}{m} \hat{u}^k + g_v \frac{T}{m} \hat{v}^k + \hat{v}^k\end{aligned}$$

or in matrix form with $X^k := (\hat{u}^k, \hat{v}^k)^t$:

$$X^{k+1} = \begin{pmatrix} 1 - 4\delta_1 n^2 \frac{T}{m} \xi_n + f_u \frac{T}{m} & f_v \frac{T}{m} \\ g_u \frac{T}{m} & 1 - 4\delta_2 n^2 \frac{T}{m} \xi_n + g_v \frac{T}{m} \end{pmatrix} X^k.$$

For the explicit Euler. For the implicit Euler method we get:

$$\begin{aligned}\hat{u}^k &= \hat{u}^{k+1} \left(1 + 4\delta_1 n^2 \frac{T}{m} \xi_n - f_u \frac{T}{m} \right) - f_v \frac{T}{m} \hat{v}^{k+1}, \\ \hat{v}^k &= \hat{v}^{k+1} \left(1 + 4\delta_2 n^2 \frac{T}{m} \xi_n - g_v \frac{T}{m} \right) - g_u \frac{T}{m} \hat{u}^{k+1}.\end{aligned}$$

So we obtain the system matrix:

$$\begin{pmatrix} 1 + 4\delta_1 n^2 \frac{T}{m} \xi_n - f_u \frac{T}{m} & -f_v \frac{T}{m} \\ -g_u \frac{T}{m} & 1 + 4\delta_2 n^2 \frac{T}{m} \xi_n - g_v \frac{T}{m} \end{pmatrix}^{-1}.$$

Last but not least, we will analyse the Crank Nicolson scheme, where:

$$\frac{u_{i,j}^{k+1} - u_{i,j}^k}{T/m} = \frac{\mathcal{U}^k + \mathcal{U}^{k+1}}{2}, \quad \frac{v_{i,j}^{k+1} - v_{i,j}^k}{T/m} = \frac{\mathcal{V}^k + \mathcal{V}^{k+1}}{2}.$$

We then get for our analysis with $Y^k := (\hat{u}^k, \hat{v}^k)^t$ that:

$$Y^{k+1} - Y^k = \frac{T}{2m} \begin{pmatrix} f_u - 4\delta_1 n^2 \xi_n & f_v \\ g_u & g_v - 4\delta_2 n^2 \xi_n \end{pmatrix} Y^k + \frac{T}{2m} \begin{pmatrix} f_u - 4\delta_1 n^2 \xi_n & f_v \\ g_u & g_v - 4\delta_2 n^2 \xi_n \end{pmatrix} Y^{k+1}$$

$$\Leftrightarrow Y^{k+1} = \left(E - \frac{T}{2m} M_n\right)^{-1} \left(E + \frac{T}{2m} M_n\right) Y^k.$$

For the methods to be stable, we need the amplification factors to be bounded by at least 1, from above. Meaning the spectral radii ρ of our system matrices have to be ≤ 1 . So we have the following system matrices:

1. explicit Euler:

$$E + \frac{T}{m} M_n,$$

with eigenvalues $\lambda = 1 + \frac{T}{m} \mu$ where μ denotes the eigenvalue of matrix M_n .

2. implicit Euler:

$$\left(E - \frac{T}{m} M_n\right)^{-1},$$

with eigenvalues $\lambda = \left(1 - \frac{T}{m} \mu\right)^{-1}$.

3. Crank Nicolson:

$$\left(E - \frac{T}{2m} M_n\right)^{-1} \left(E + \frac{T}{2m} M_n\right),$$

with eigenvalues $\lambda = \left(1 - \frac{T}{2m} \mu\right)^{-1} \left(1 + \frac{T}{2m} \mu\right)$.

For stability we then need $\rho \leq 1 \Leftrightarrow |\lambda| \leq 1$. So we have the following conditions for each scheme:

1) explicit Euler: $\left|1 + \frac{T}{m} \mu\right| \leq 1$

2) implicit Euler: A-stable

3) Crank Nicolson: A-stable.

3 Implementation & Architecture

The numerical simulation of the Gray-Scott reaction-diffusion system was implemented using a highly modular programming approach. The software is written entirely in Python, utilizing standard scientific libraries such as NumPy and SciPy to handle 2D array operations, spatial discretization, and sparse linear algebra.

3.1 Software Architecture, Modularity, and Quality Assurance

To ensure the code is clean, maintainable, and easy to test, the project is structurally partitioned into specific sub-modules. This design strictly follows standard Software Engineering (SE) practices:

- **User Interface & Execution (`interface.py` & `run_sim.py`):** This acts as the control hub of the project. It provides a unified function (`run_simulation`) that allows users to easily switch between different biological patterns (leopard or giraffe) and numerical methods (explicit or implicit) without modifying the core logic.
- **Models (`models.py`):** This module defines the core Gray-Scott reaction equations. It also serves as a parameter dictionary, storing the specific diffusion coefficients (D_u, D_v) and reaction rates (F, K) required to generate different animal fur patterns.
- **Seeding Strategies (`seeding/`):** The pattern formation is highly sensitive to initial conditions. This module manages the starting states, providing either localized concentrated dots (for the leopard) or global uniform noise (for the giraffe) to push the system out of its homogeneous equilibrium.
- **Numerical Solvers (`solvers/`):** These are the mathematical engines of the project. We implemented two distinct integration classes: the `ExplicitScheme` based on the Forward Euler method, and the other one, `CrankNicolsonScheme`, is for unconditionally stable implicit solving. These base solvers also lay the groundwork for our advanced GPU and Matrix-Free optimizations.
- **Visualization (`visual.py`):** A dedicated rendering module that safely decouples data generation from image plotting. It converts the final 2D concentration arrays into clear grayscale heatmaps to visually track the pattern evolution.
- **Quality Assurance & Convergence Validation (`unittest.py` & `validate_order.py`):** To mathematically guarantee the reliability of our code, we built an automated testing suite. While `unittest.py` checks the basic accuracy of the Laplacian operator and time steps, `validate_order.py` rigorously proves our numerical accuracy. By using the Method of Manufactured Solutions (MMS) and self-convergence tests, this script automatically plots and verifies that our shared spatial discretization achieves strict $\mathcal{O}(\Delta x^2)$ accuracy, while the explicit and implicit temporal integrations perfectly match their theoretical $\mathcal{O}(\Delta t)$ and $\mathcal{O}(\Delta t^2)$ convergence orders.
- **2D-Explicit/:** Contains scripts (e.g., `Leo_Explicit_solver.py` and `Gira_Explicit_Solver.py`) dedicated to the Forward Euler method. These scripts rely exclusively on NumPy for spatial discretization and vector arithmetic.
- **Crank-Nicolson-implicit/:** Contains scripts (e.g., `Leo_CN_Solver.py` and `Gira_CN_Solver.py`) that implement the unconditionally stable Crank-Nicolson method, utilizing SciPy's sparse matrix libraries to construct and solve the resulting large-scale linear systems.

Anatomy of a Monolithic Script:

Instead of relying on a shared library or object-oriented components, each script in this baseline is completely self-contained. The execution follows a strict, top-to-bottom procedural workflow:

1. **Parameter Hardcoding:** Physical parameters specific to the target animal (diffusion rates D_u, D_v and reaction coefficients F, K) alongside numerical parameters (grid size N , spatial step dx , and time step Δt) are defined as global variables at the top of the file.
2. **Grid Allocation & Initialization:** NumPy arrays are pre-allocated for the concentration fields u and v . The distinct initial seeding logic (either localized spots for the leopard or global noise for the giraffe) is directly applied to these arrays via matrix slicing and built-in random generators.
3. **Temporal Integration Loop:** A primary for-loop handles the time-stepping. Within this loop, spatial derivatives (Laplacians) and reaction terms are computed, and the state arrays are iteratively overwritten.
4. **Inline Visualization:** Instead of decoupling the data generation from post-processing, `matplotlib.pyplot` is invoked directly at the end of the script (and occasionally within the loop for monitoring) to render and display the 2D concentration heatmaps.

This uncoupled architecture makes the mathematical operations highly transparent for algorithmic verification, though it inherently violates the DRY (Don't Repeat Yourself) principle, establishing a raw mathematical baseline that prioritizes theoretical clarity over software optimization.

3.2 Baseline Codebase Structure and Workflow

To study the mathematical differences between the numerical methods without adding complex software layers, our initial baseline uses standalone Python scripts. We grouped the code into two main folders based on the time-stepping method:

- **2D-Explicit/:** Contains scripts (e.g., `Leo_Explicit_solver.py` and `Gira_Explicit_Solver.py`) that use the Forward Euler method. These scripts rely on NumPy for basic array math and spatial calculations.
- **Crank-Nicolson-implicit/:** Contains scripts (e.g., `Leo_CN_Solver.py` and `Gira_CN_Solver.py`) that use the unconditionally stable Crank-Nicolson method. They use SciPy's sparse matrix tools to build and solve large linear systems.

Structure of a Baseline Script:

Instead of using shared classes or complex structures, each script runs independently from top to bottom in a clear, step-by-step process:

1. **Parameter Setup:** Physical parameters (diffusion rates D_u, D_v and reaction coefficients F, K) and numerical settings (grid size N , spatial step dx , and time step Δt) are defined directly as global variables at the top of the file.
2. **Grid Initialization:** We create NumPy arrays for the concentration fields u and v . The starting patterns—either small center spots for the leopard or global random noise for the giraffe—are added directly to these arrays.

3. **Time-Stepping Loop:** A main `for`-loop moves the simulation forward in time. Inside this loop, we calculate the spatial derivatives (Laplacians) and reaction equations, updating the grid at each step.
4. **Direct Visualization:** Rather than saving the data to plot later, `matplotlib.pyplot` is used directly at the end of the script to draw the 2D heatmaps of the patterns.

This straightforward approach makes the mathematical operations highly transparent and easy to verify. While it duplicates some code (violating the DRY principle), it provides a solid mathematical baseline that focuses on theoretical clarity before introducing advanced software optimizations.

3.3 Explicit Solver and Vectorization

The Explicit Euler scheme computes the future state of the system step by step. The unoptimized implementation translates the mathematical Forward Euler formula directly into Python code. Because this explicit method is strictly limited by the Courant-Friedrichs-Lewy (CFL) stability condition, it requires extremely small discrete time steps (Δt) to prevent the simulation from crashing (numerical divergence).

Spatial Discretization via `numpy.roll`:

The 2D discrete Laplacian (∇^2) is calculated using a standard 5-point central difference stencil. To avoid the slow performance of standard `for`-loops in Python, this operation is fully vectorized. We defined a separate `laplacian_2d(field)` function that utilizes `numpy.roll`. In math, shifting the array by one index maps perfectly to the spatial neighbors:

$$\nabla^2 Z_{i,j} \approx \frac{Z_{i+1,j} + Z_{i-1,j} + Z_{i,j+1} + Z_{i,j-1} - 4Z_{i,j}}{(\Delta x)^2} \quad (1)$$

In the code, this translates precisely to:

```
(np.roll(Z, 1, axis=0) + np.roll(Z, -1, axis=0) + np.roll(Z, 1, axis=1) + np.roll(Z, -1, axis=1) - 4 * Z) / dx2.
```

A major benefit of using `np.roll` is that it naturally handles Periodic Boundary Conditions. When elements are shifted past the edge of the array, they simply wrap around to the opposite side. This eliminates the need to create extra "ghost cells" or pad the grid.

Memory Bandwidth Bottleneck:

While these vector additions and shift operations look clean in code, this basic NumPy implementation suffers from a classic "Memory Bandwidth Bottleneck". Every single `numpy.roll` operation and math operator (`+`, `-`, `*`) creates a new temporary array in the system memory. For a standard 200×200 grid, calculating just one Laplacian generates multiple 40,000-element arrays. Because the CPU must constantly move these large arrays between the main RAM and the L1/L2 cache, the execution speed becomes heavily memory-bound.

3.4 Sparse Implicit Solver (Crank-Nicolson)

To overcome the strict time step limits of the explicit method, we implemented the implicit Crank-Nicolson (CN) scheme. This semi-implicit method handles the linear diffusion parts using an average (trapezoidal rule), but calculates the nonlinear reaction term $u \cdot v^2$ explicitly to avoid complex Newton-Raphson iterations.

Kronecker Product for 2D Matrix Construction:

Setting up the CN method turns the 2D spatial Laplacian into a large sparse matrix. In our unoptimized script,

the construction process is highly straightforward:

- a) A 1D differentiation matrix D_{1D} is created using `scipy.sparse.spdiags` with diagonals $[1, -2, 1]$.
- b) Periodic boundary conditions are handled by placing 1s at the outer diagonals of the 1D matrix (offsets $-N+1$ and $N-1$).
- c) The final 2D Laplacian operator L (a $40,000 \times 40,000$ matrix for $N = 200$) is built using the Kronecker product (`scipy.sparse.kron`) of D_{1D} with the identity matrix I_{1D} :

$$L = \frac{1}{(\Delta x)^2} ((I \otimes D_{1D}) + (D_{1D} \otimes I)) \quad (2)$$

System Formulation and Direct Solving:

The CN scheme sets up the equation as a solvable linear system $Ax = b$. Two setup matrices are built for both the u and v fields: $A_u = I - \frac{\Delta t}{2} D_u L$ (the implicit left-hand side) and $B_u = I + \frac{\Delta t}{2} D_u L$ (the explicit right-hand side). At every time step, the right-hand side vector `rhs_u` is computed by multiplying B_u with the current state and adding the reaction terms.

A significant performance bottleneck in this unoptimized version comes from the solving strategy and data structure overhead. First, instead of an iterative approach, the script relies on SciPy’s direct solver (`scipy.sparse.linalg.factorized`) to perform an LU decomposition on the sparse matrix. While unconditionally stable, this consumes a massive amount of memory and time. Second, the 2D physical grid arrays (200×200) must be continuously flattened into 1D vectors (`u.flatten()`) to be processed by the linear solver. After the solver finds the solution for the next step, these vectors must be immediately reshaped back to 2D (`reshape((N, N))`) to evaluate the nonlinear term $u \cdot v^2$. This constant flattening and reshaping cycle adds noticeable structural overhead.

3.5 Biological Initialization and Seeding Strategies

The final pattern generated by the Gray-Scott model depends heavily on its initial state. Instead of using complex setup functions, our unoptimized scripts directly manipulate the NumPy arrays using random masks and noise generators:

- **Leopard Pattern (Global Random Seeding):** We start with a uniform, unreacted background where $u = 1.0$ and $v = 0.0$. To trigger the reaction, we use a global seeding strategy. A random mask (`np.random.rand > 0.98`) selects approximately 2% of the pixels as "reaction centers," where we inject an initial perturbation by setting both u and v to 0.5. Finally, we add a small Gaussian noise (with a scale of 0.02) across the entire grid to break perfect symmetry. These randomly scattered seeds trigger isolated reactions that grow outward and form the leopard spots.
- **Giraffe Pattern (Global Unstable Perturbation):** Creating a connected network of patches requires a different approach. We initialize the entire grid near an unstable equilibrium point, setting the base values to $u = 0.5$ and $v = 0.3$. We then apply uniform random noise (ranging from -0.1 to 0.1) to every single point. To keep the concentrations physically valid, we use `np.clip` to ensure all values stay strictly between 0 and 1. By pushing the entire domain out of balance simultaneously, the reactions happen everywhere at once. This prevents isolated dots from forming and forces the reacting regions to merge into the large, connected shapes typical of a giraffe coat.

4 Parallelization

4.1 GPU-Accelerated Explicit Reaction-Diffusion System Solver based on CuPy

4.1.1 Introduction and Purpose

In simulating the dynamic evolution of Turing patterns (e.g., leopard spots and giraffe patches), the Explicit Euler method typically requires extremely small time steps (Δt) and a large number of iterations (e.g., 50,000 steps) to remain stable. Under traditional CPU architectures, computing these points one by one consumes a significant amount of time.

To solve this, our explicit solver introduces a GPU-accelerated architecture driven by CuPy. It overcomes computational bottlenecks using fine-grained parallelization. Its primary applications and acceleration principles include:

- **Massive Data-Level Parallelism:** We map every pixel of the $N \times N$ grid to independent GPU threads, allowing the entire global state to be updated simultaneously.
- **Bypassing Python Overhead & Optimizing Data Flow:** By directly running low-level C++/CUDA code via CuPy's `RawKernel`, and strictly limiting data transfers between the CPU (Host) and GPU (Device), the system can generate complex animal patterns in real-time.

4.1.2 Mathematical Principles and Parallelization Mechanisms

This GPU architecture is built on the following mathematical and hardware foundations:

A. Central Difference Discretization in the Spatial Domain To compute the diffusion term, the system applies a second-order central difference approximation of the Laplacian operator at each grid point (i, j) :

$$\Delta u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2} \quad (3)$$

Inside the GPU kernel, modulo operations (%) are used for wrap-around coordinate mapping. This perfectly satisfies the Periodic Boundary Conditions directly within the algorithm.

B. Explicit Euler Integration in the Time Domain Using the concentrations and reaction rates at the current time step (k) , the system predicts the next state $(k + 1)$ using linear extrapolation:

$$u^{k+1} = u^k + \Delta t \cdot \left(D_u \cdot \Delta u^k + f(u^k, v^k) \right) \quad (4)$$

Because all variables on the right side are already known at step k , the computation for each spatial grid point is completely independent. This provides the perfect mathematical setup for the SIMT (Single Instruction, Multiple Threads) architecture used by modern GPUs.

C. Thread Hierarchy and Mapping In the parallel implementation, computing tasks are divided into Grids, Blocks, and Threads. For example, on a 200×200 grid, we set our thread blocks to 16×16 , meaning each Block runs 256 concurrent threads.

To cover the entire grid, the Grid dimension is calculated as $\lceil 200/16 \rceil \times \lceil 200/16 \rceil = 13 \times 13$ Blocks. Each thread finds its unique global coordinate (x, y) using built-in CUDA variables (e.g., `threadIdx.x + blockIdx.x * blockDim.x`), ensuring every single pixel is handled by its own independent thread.

D. Memory Access Optimization and Data Flow Another major reason for the speedup is how we handle data:

- **Using Single Precision for Faster Processing:** Physical constants and arrays are explicitly cast to `float32`. On GPUs, single-precision calculations double the register efficiency and significantly boost the floating-point operations per second (FLOPS).
- **Minimizing Host-Device Transfers:** Moving data back and forth over the PCIe bus is a common bottleneck. Our code copies data from the Host Memory to the Device Memory only once at the beginning. After that, tens of thousands of iterations run entirely inside the VRAM. The final images are sent back to the Host only when the simulation finishes, eliminating slow communication delays.

4.1.3 Code Architectural Analysis

Our parallel acceleration scheme is structured into the following five core modules:

Module Name	Parallelization Strategy & Function	Core Logic & Mathematical Equivalence
Parameter & VRAM Initialization	GPU Setup: Converts values to <code>float32</code> and copies data from Host to Device once.	Pre-allocates empty arrays of the same size in VRAM (<code>empty_like</code>) to prepare for zero-copy updates.
RawKernel Definition	Low-Level Logic: Uses injected C code for the math equations.	SIMT Scheduling: All threads run the same instructions but process different spatial pixels.
Thread Grid Setup	Resource Mapping: Sets <code>block_size</code> to 16×16 .	Uses ceiling division to ensure the Grid contains enough threads to cover the entire $N \times N$ matrix.
Synchronization	Temporal Consistency: Controls the order of kernel executions.	GPU Sync: The GPU naturally finishes computing all N^2 threads for the current time step before starting the next one, preventing data mixing.
Efficient Pointer Swapping	Zero-Copy Updates: Swaps variable names at the Python level.	Swaps the memory addresses of <code>u_gpu</code> and <code>u_next_gpu</code> . This instantly moves the simulation forward without copying heavy $\mathcal{O}(N^2)$ memory arrays.

4.1.4 Conclusion

The CuPy-based explicit architecture strictly translates the mathematical model of the reaction-diffusion system into high-performance code. From an engineering perspective, it eliminates traditional computing bottlenecks by mapping threads intelligently and keeping all data inside the VRAM. This parallel design reliably supports different model requirements—handling both the fast reactions of the leopard model ($dt = 1.0$, $D_u = 0.16$) and the long-term, strict steps of the giraffe model ($dt = 0.02$, $D_u = 0.8$). By offloading the heavy math to the GPU, our system creates complex biological patterns exponentially faster than standard CPU methods.

4.2 Solving the Reaction-Diffusion System using a Matrix-Free Implicit Strategy

4.2.1 Introduction and Purpose

In biological pattern simulations (e.g., leopard spots and giraffe patches), establishing a large 2D grid ($N \times N$) is required to achieve high visual resolution. While the traditional implicit Crank-Nicolson (CN) method offers unconditional stability and allows for larger time steps, its main challenge lies in solving a massive linear system $Ax = b$ at each step.

For a 200×200 grid, the coefficient matrix A reaches a dimension of $40,000 \times 40,000$. To solve this, our project introduces a Matrix-Free architecture. Its primary algorithmic advantages include:

- **Memory Minimization:** Traditional direct solvers (e.g., LU factorization) require explicitly building and storing an $N^2 \times N^2$ sparse matrix. The matrix-free strategy skips this process entirely. By storing only the $N \times N$ concentration field, it avoids massive memory overhead and strictly bounds the spatial complexity to $\mathcal{O}(N^2)$.
- **Computational Efficiency:** Coupled with the Conjugate Gradient (CG) method, the system avoids matrix inversion operations that can cost up to $\mathcal{O}(N^3)$. Instead, it efficiently calculates the next physical state using a low-overhead function mapping.
- **Parallel Potential:** The matrix-free architecture avoids the complex memory addressing issues of sparse matrix inversions. This simplifies the computational logic and provides a solid foundation for vectorization and parallel acceleration.

4.2.2 Mathematical Principles

This architecture is based on three core mathematical concepts:

A. Operator Definition In linear algebra, a matrix-vector multiplication Ax is essentially a linear transformation $\mathcal{A} : \mathbb{R}^{N^2} \rightarrow \mathbb{R}^{N^2}$. In the CN scheme, the operator on the left-hand side is:

$$\mathcal{A} = I - \frac{\Delta t \cdot D}{2} \nabla^2 \quad (5)$$

where I is the identity operator and ∇^2 is the Laplacian. The matrix-free architecture directly programs this mapping function instead of storing every element of $\mathcal{A}$.

B. Spatial Discretization Using a second-order central difference to approximate the Laplacian, the calculation at a 2D grid point (i, j) is:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2} \quad (6)$$

The code uses `numpy.roll` to perform circular shifts. This naturally applies periodic boundary conditions by treating the grid as a continuous, wrap-around space.

C. Convergence and the Conjugate Gradient Method Using the Conjugate Gradient (CG) method to solve the implicit step $Ax = b$ is a highly effective choice. Because the discretized operator $\mathcal{A}$ forms a Symmetric Positive Definite (SPD) matrix, the CG algorithm converges very reliably.

At each iteration, CG only requires basic “matrix-vector multiplications” (which perfectly matches our matrix-free mapping function) without any matrix factorization. Furthermore, since the physical time step Δt is extremely small, the system changes very little between steps. By using the current concentration field u_n as the initial guess (x_0) for the CG solver, the algorithm requires very few iterations to reach a high tolerance (e.g., `rtol=1e-5`), leading to exceptionally fast convergence.

4.2.3 Code Architectural Analysis

This optimized code is divided into five functional modules. Notably, this architecture demonstrates excellent CPU parallel execution capabilities through vectorized operations.

Module Name	Function Description	Core Logic & Mathematical Equivalence
Parameter & Operator Setup	Defines the physical parameters and domain.	Uses <code>LinearOperator</code> to define mapping rules, treating the operation as an (N^2, N^2) matrix.
Spatial Laplacian Calculation	Computes the spatial derivatives.	Operates directly on 2D arrays, avoiding the complex index tracking needed for flattened vectors.
Operation Rule Definition	Constructs the mathematical sides of the equation.	Implements the mapping $x \mapsto (I \pm \frac{\Delta t D}{2} \nabla^2)x$. It relies heavily on vectorized operations to boost CPU efficiency.
Implicit Time-Stepping Solver	Executes single-step updates.	Uses the Conjugate Gradient (CG) method to solve $Ax = b$. It speeds up convergence by using the current state as the initial guess.
Evolution Main Loop	Handles time-domain recursion.	Combines the <code>Grey_Scott</code> reaction terms with the implicit diffusion operator.

Parallelization Potential: Although this implementation runs on the CPU, the matrix-free architecture has strong parallelization potential. The core Laplacian calculation uses `numpy.roll`, completely avoiding slow scalar `for`-loops. This makes it a standard **vectorized operation** that can directly benefit from the SIMD (Single Instruction, Multiple Data) instructions of modern CPUs (like AVX-512). This hardware-level acceleration makes it much faster than traditional sparse matrix traversal.

4.2.4 Conclusion

By rewriting the implicit CN method into a “matrix-free” format, the system keeps the stability benefits of the implicit method while avoiding the massive $N^2 \times N^2$ sparse matrix assembly. It restricts the memory usage strictly to $\mathcal{O}(N^2)$ and avoids the slow $\mathcal{O}(N^3)$ processing time of direct solvers. Additionally, pairing it with the CG algorithm and a smart initial guess significantly speeds up convergence. This architecture reliably and efficiently supports the long-term simulations needed for both the leopard model ($D_u = 0.16, D_v = 0.08$) and the giraffe model ($D_u = 0.8, D_v = 0.4$).

5 Results & Performance

5.1 Numerical Convergence and Accuracy Validation

To rigorously verify the mathematical correctness of our code, we validated the convergence orders of our numerical methods before generating the biological patterns. Since the full non-linear Gray-Scott model lacks a simple analytical solution, we decoupled the spatial and temporal integration schemes for independent testing.

Spatial Convergence (Laplacian Operator):

Using the Method of Manufactured Solutions (MMS) with a known test function $U(x, y) = \sin(2\pi x) \cos(2\pi y)$, we measured the maximum absolute error (L_∞ norm) of our 5-point central difference implementation across different spatial grid sizes (Δx). As shown in the left panel of Figure 1, the numerical error drops perfectly parallel to the theoretical $\mathcal{O}(\Delta x^2)$ reference line. We calculated a fitted slope of exactly 2.00, proving that our shared spatial discretization guarantees strict second-order accuracy.

Temporal Convergence (Full Mode vs. Pure Diffusion):

In our implicit solver (`Implicit.py`), we employ the semi-implicit approach: the diffusion terms are treated implicitly using Crank-Nicolson to ensure unconditional stability, while the nonlinear reaction term is treated explicitly to avoid computationally expensive Newton-Raphson iterations. By mathematical definition, the global temporal accuracy of this semi-implicit scheme is dominated by the first-order explicit reaction step.

We performed a self-convergence test using a highly refined reference solution ($\Delta t = 0.005$). The right panel of Figure 1 tracks the errors across different time steps (Δt). As expected, both the Explicit Euler (blue line) and the IMEX Crank-Nicolson (green line) for the full model show a slope of approximately 1.0, matching the theoretical $\mathcal{O}(\Delta t)$ first-order accuracy.

To specifically validate the correct implementation of our implicit Crank-Nicolson architecture, we isolated the diffusion operator by temporarily turning off the reaction kinetics. As the orange line demonstrates, once stripped of the explicit reaction interference, our implicit diffusion core perfectly achieves its theoretical $\mathcal{O}(\Delta t^2)$ second-order accuracy with a slope of 2.00.

This rigorous testing strategy confirms that our numerical foundation is highly reliable, while the semi-implicit design choice deliberately balances computational speed with structural stability.

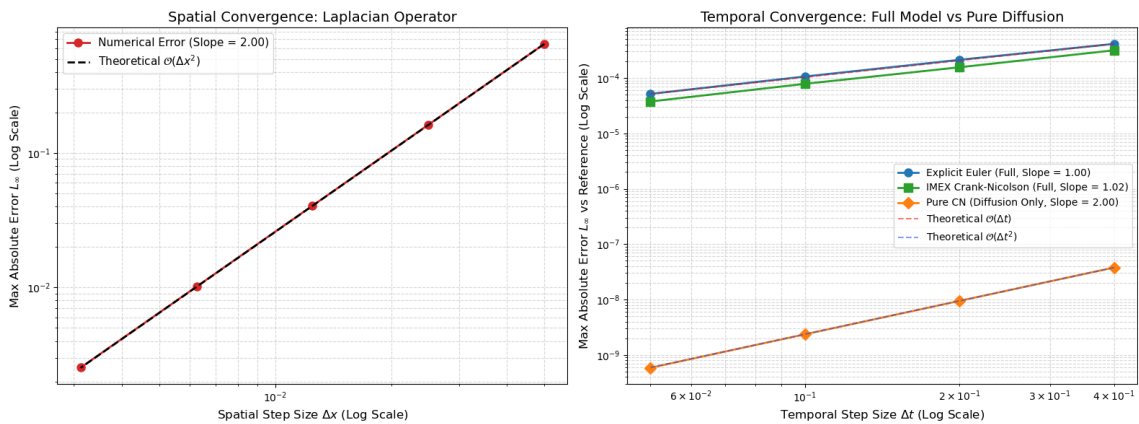


Figure 1: Log-log plots validating the spatial convergence order of the Laplacian operator (left) and the temporal convergence orders comparing the full IMEX model against the pure diffusion Crank-Nicolson core (right).

5.2 Pattern Generation Analysis

Both numerical solvers successfully produced macro-scale biological patterns by integrating the Gray-Scott kinetics over a 200×200 spatial domain.

The simulations demonstrated robust pattern formation under different parameter regimes:

- **Leopard Coat Formation:** Utilizing $F = 0.0367$, $K = 0.0649$, $D_u = 0.16$, and $D_v = 0.08$ with sparse 2% localized seeds, the system successfully broke spatial symmetry to form stable, isolated spots characteristic of a leopard.
- **Giraffe Patch Formation:** Utilizing $F = 0.089$, $K = 0.060$, and higher diffusion rates ($D_u = 0.8$, $D_v = 0.4$) alongside global uniform noise near an unstable equilibrium, the system produced wide, dark patches separated by narrow light corridors. The "merging mechanism" allowed initial perturbations to dynamically combine into a clean polygonal grid characteristic of a giraffe.

5.3 Performance and Algorithm Benchmarking Before Optimization

Before introducing advanced parallelization, we developed an automated profiling suite (`benchmark_profile.py`) to evaluate the baseline performance of the Explicit Euler and implicit Crank-Nicolson solvers. These tests highlight the core bottlenecks when processing high-resolution reaction-diffusion systems.

The following table outlines the numerical setup utilized for these evaluations.

Table 1: Numerical setup for benchmark evaluations (200×200 grid resolution).

Target Pattern	Solver Architecture	Total Steps	Time Step (Δt)	Stability Limit
Leopard	CPU Explicit Euler	10,000	1.0	CFL Constrained
Leopard	Implicit CN Solver	10,000	1.0	Unconditionally Stable
Giraffe	CPU Explicit Euler	50,000	0.02	CFL Constrained
Giraffe	Implicit CN Solver	50,000	0.02	Unconditionally Stable

Analysis of Parameter Selection: Stiffness and Stability

As shown in Table 1, there is a significant difference in the time step (Δt) and the total number of steps between the two animal models. This selection is governed by the **stiffness** of the partial differential equations, which is determined by the reaction parameters (F , K) and diffusion rates (D_u , D_v):

- **Leopard Model (Low Stiffness):** The reaction kinetics for the leopard pattern are relatively slow and gentle. With a lower diffusion rate ($D_u = 0.16$), the system state changes gradually. This allows us to use a large time step ($\Delta t = 1.0$) without breaking the stability limits of the explicit solver. To reach the fully formed state where spots are clearly defined, a total physical time of 10,000 units is required, which we achieve in exactly 10,000 steps.
- **Giraffe Model (High Stiffness):** The giraffe model is much more highly dynamic due to its higher diffusion rates ($D_u = 0.8$) and reaction coefficients. The system state changes rapidly, making the equations numerically "stiff." If we used a large step size like $\Delta t = 1.0$, the explicit solver would immediately suffer from numerical divergence (or "blow up") because it violates the Courant-Friedrichs-Lewy (CFL) stability condition. To maintain a stable simulation, we are forced to reduce the step size to $\Delta t = 0.02$.

Total Iterations and Physical Time

The total number of steps is a direct consequence of the required "Physical Time" ($T_{total} = \Delta t \times \text{Steps}$) needed for the biological patterns to fully develop:

- The leopard pattern matures over a physical duration of 10,000. At $\Delta t = 1.0$, this requires 10,000 iterations.
- The giraffe pattern reaches maturity much faster in terms of physical time (around 1,000 units). However, because the high stiffness forces us to use a tiny step size of $\Delta t = 0.02$, the system must execute $1,000/0.02 = 50,000$ iterations to complete the simulation.

This comparison explains the core motivation for our subsequent optimizations. While the implicit CN method is unconditionally stable, the massive amount of iterations required by the explicit method for stiff systems like the giraffe makes GPU acceleration essential for practical performance.

Execution Time Analysis (Refer to Figure 2):

The generation of complex Turing patterns clearly exposes the performance limits of the unoptimized baseline implementations. The following analysis outlines the runtime characteristics and implementation shortcomings of the pure CPU-based solvers before advanced parallelization, specifically focusing on the time required to generate the final simulation images.

In the unoptimized `explicit.py`, the explicit Euler method uses standard second-order central differences for spatial discretization. For stiff simulations like the Giraffe pattern, satisfying the strict Courant-Friedrichs-Lewy (CFL) stability condition requires a very small time step (e.g., $\Delta t = 0.02$), forcing the system to run 50,000 iterations. Although this pure NumPy implementation is mathematically intuitive, the massive serial time-stepping loop is heavily bottlenecked by the Python Global Interpreter Lock (GIL) and constant dynamic array allocations. As a result, generating the final visual results takes a significant amount of time.

Conversely, the unoptimized `Implicit.py` relies on SciPy's sparse matrix pre-factorization and direct solving methods (`splinalg.factorized`). While this approach mathematically provides unconditional stability and removes the explicit time-step limits, its actual implementation requires building a huge 2D Laplacian sparse matrix. The computational cost of direct matrix inversion, combined with the constant overhead of reshaping data arrays (`flatten` and `reshape` at every step for non-linear evaluations), greatly increases the per-iteration complexity. At identical spatial resolutions, this method struggles with high memory usage and slow single-step execution, making the overall image generation exceptionally slow.

The visual representation of these overall benchmark comparisons is provided in the figure below.

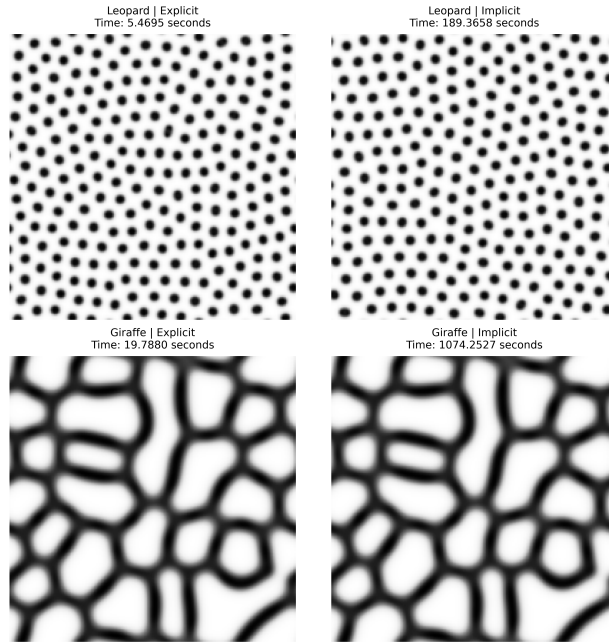


Figure 2: Automated benchmark profiling comparing the total execution time (in seconds) of Explicit Euler and implicit Crank-Nicolson solvers.

Execution Time Analysis (Refer to Table 2): In the unoptimized `explicit.py` implementation, the spatial Laplacian operator relies exclusively on the `numpy.roll` function. This computes the second-order central difference on the 2D grid while naturally handling periodic boundary conditions. However, for simulation tasks that require a highly refined time step (e.g., $\Delta t = 0.02$, demanding 50,000 iterations), this pure NumPy baseline shows severe performance drops.

The main problem lies in its excessive memory bandwidth consumption. Due to the Python GIL and the lack of low-level memory pre-allocation, every `np.roll` call and basic math operation forces the CPU to allocate large temporary arrays in the main memory. Over tens of thousands of iterations, the CPU must continuously move huge amounts of data between its high-speed cache (L1/L2) and the system RAM. This massive data movement turns a compute-bound algorithm into a heavily memory-bound task, resulting in very long execution times.

Table 2: cProfile Results for Unoptimised CPU Explicit Solver (Top Functions)

Function / Module	Calls	Total Time (s)	Per Call (s)	Cum. Time (s)
explicit.py: laplacian_2d()	100,000	6.964	0.000	12.855
numeric.py: roll()	400,000	5.393	0.000	5.862
explicit.py: step()	50,000	4.179	0.000	17.034
models.py: Grey_Scott()	50,000	3.203	0.000	3.203
explicit.py: run()	1	0.973	0.973	21.217
numeric.py: normalize_axis_tuple()	400,000	0.276	0.000	0.350
multiarray.py: empty_like()	400,000	0.027	0.000	0.027
builtins.print	51	0.007	0.000	0.007
seeding.py: giraffe_seeding()	1	0.001	0.001	0.001
interface.py: run_simulation()	1	0.000	0.000	21.219

Execution Time Analysis (Refer to Table 3): Before implementing the Matrix-Free Conjugate Gradient optimization, we profiled the unoptimized Crank-Nicolson (CN) baseline solver. This implementation uses SciPy’s sparse matrix structures and a direct solver based on SuperLU (`splinalg.factorized`). While providing unconditional stability, it suffers from significant memory and time overheads. For an $N \times N$ grid, explicitly building an $N^2 \times N^2$ sparse matrix rapidly consumes system memory. Additionally, continuously flattening and reshaping 2D physical fields into 1D vectors for matrix-vector multiplication introduces a constant data structure overhead, preventing efficient scaling to high-resolution grids.

The profiling results clearly illustrate the severe time bottlenecks of this direct solving scheme. For a simulation of 50,000 physical time steps, the total wall-clock time reached 1062.829 seconds (approximately 17.7 minutes). The computational workload is highly unbalanced: over 95% of the execution time (1010.917 seconds) was spent entirely in the solve method of the SuperLU objects. Even though the sparse coefficient matrix is pre-factorized (LU Factorization) during setup, solving the linear system at each step for large dimensions (e.g., a 40,000-dimensional vector for a 200×200 grid) remains computationally expensive.

Beyond the direct solver bottleneck, sparse matrix-vector multiplication (`csc_matvec`) and its dot operations were the second largest overhead, consuming approximately 40 seconds cumulatively. In stark contrast, the core physical kinetics evaluations (`Grey_Scott` function) and the internal time-stepping logic accounted for less than 18 seconds combined.

In summary, while the traditional sparse matrix CN scheme removes CFL constraints, its direct inversion strategy requires too much memory and causes major execution delays. This strongly highlights the need to adopt an iterative, matrix-free architecture.

Table 3: cProfile Results for Unoptimized Implicit Solver (Top Functions)

Function / Module	Calls	Total Time (s)	Per Call (s)	Cum. Time (s)
SuperLU objects: solve()	100,000	1010.917	0.010	1010.917
_sparsetools.csc_matvec	100,000	17.875	0.000	17.875
models.py: Grey_Scott()	50,000	9.110	0.000	9.110
Implicit.py: step()	50,000	8.213	0.000	1046.383
Implicit.py: run()	1	4.387	4.387	1059.888
numeric.py: isscalar()	200,010	0.412	0.000	1.119
_base.py: dot()	100,000	0.412	0.000	23.040
builtins.isinstance	500,207	0.383	0.000	0.717
_abc._abc_instancecheck	200,005	0.244	0.000	0.244
_base.py: _matmul_dispatch()	100,000	0.231	0.000	21.066

5.4 Performance and Algorithm Benchmarking Following Parallelization Optimization

Using the automated profiling suite (`benchmark_profile.py`), we compared the execution times of the GPU Explicit Euler against the CPU Matrix-Free CN solvers.

The following table outlines the numerical setup utilized for these benchmark evaluations.

Table 4: Numerical setup utilized in the benchmark suite (200×200 grid resolution).

Target Pattern	Solver Architecture	Total Steps	Time Step (Δt)	Stability Limit
Leopard	GPU Explicit Euler (CuPy)	10,000	1.0	CFL Constrained
Leopard	Matrix-Free Implicit CN (SciPy)	10,000	1.0	Unconditionally Stable
Giraffe	GPU Explicit Euler (CuPy)	50,000	0.02	CFL Constrained
Giraffe	Matrix-Free Implicit CN (SciPy)	50,000	0.02	Unconditionally Stable

Execution Time Analysis (Refer to Figure 3):

The benchmark results clearly illustrate the operational trade-offs between the two architectures. For the Giraffe simulation—which is highly stiff and requires a very small time step ($\Delta t = 0.02$), resulting in 50,000 iterations—the CuPy-accelerated explicit solver greatly reduces the total execution time. It achieves this by bypassing the Python Global Interpreter Lock (GIL) and leveraging massive parallelization via custom CUDA kernels.

Conversely, the Matrix-Free Crank-Nicolson method provides excellent unconditional stability. By evaluating both solvers under identical conditions (50,000 steps, $\Delta t = 0.02$), the benchmark highlights the per-iteration computing cost of the CG solver. While doing an iterative solve is computationally more expensive than a simple GPU vector addition, the CN scheme avoids the memory limits of direct matrix inversion and remains highly resilient against numerical crashes. This makes it highly competitive for CPU-bound systems or simulations too large to fit into GPU memory.

The visual representation of these benchmark comparisons is provided in the figure below.

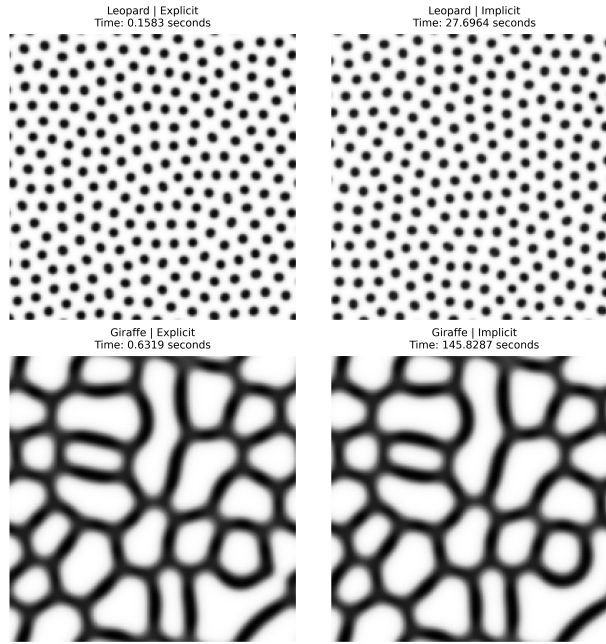


Figure 3: Automated benchmark profiling comparing the total execution time (in seconds) of CuPy-accelerated Explicit Euler vs. Matrix-Free Crank-Nicolson solvers.

Execution Time Analysis (Refer to Table 5): The profiling results clearly show the high computational throughput of the CuPy-accelerated architecture. For the Giraffe pattern simulation (50,000 iterations), standard CPU processing usually encounters severe performance bottlenecks. However, our benchmark data shows that the total wall-clock time for this GPU simulation is a mere 0.650 seconds.

Specifically, the run function in `explicit.py` takes an absolute execution time (`totttime`) of 0.640 seconds, with the only other measurable delay coming from terminal printing (`print` consuming 0.009 seconds). This proves that by using a custom CUDA kernel (`RawKernel`) and mapping operations directly to the GPU, we successfully bypassed the Python GIL and CPU memory bandwidth limits. Additionally, the near-zero execution time of data transfer functions (like `asnumpy` or `get`) confirms the efficiency of keeping all state updates directly inside the GPU VRAM (zero-copy).

Detailed profiling metrics for the GPU-accelerated explicit solver are presented in the following table.

Table 5: cProfile Results for GPU-Accelerated Explicit Solver (Top Functions)

Function / Module	Calls	Total Time (s)	Per Call (s)	Cum. Time (s)
<code>explicit.py: run()</code>	1	0.640	0.640	0.649
<code>builtins.print</code>	50	0.009	0.000	0.009
<code>explicit.py: __init__()</code>	1	0.000	0.000	0.001
<code>interface.py: run_simulation()</code>	1	0.000	0.000	0.650
<code>seeding.py: giraffe_seeding()</code>	1	0.000	0.000	0.000
<code>cupy._core.core.array</code>	2	0.000	0.000	0.000
<code>cupy.__init__.py: asnumpy()</code>	1	0.000	0.000	0.000

Execution Time Analysis (Refer to Table 6): The profiling results detail the performance distribution of the Matrix-Free Crank-Nicolson (CN) implicit solver. Over a total wall-clock time of 174.378 seconds, the system executed 50,000 physical time steps (shown by the 50,000 calls to the `Grey_Scott` reaction term).

The data reveals that the primary computational workload lies in the iterative solving process. SciPy's Conjugate Gradient (`cg`) method accounts for a cumulative time (`cumtime`) of 108.235 seconds. Because of the Matrix-Free architecture, the expensive sparse matrix-vector multiplication is replaced by dynamic Laplacian operator calculations (`_apply_laplacian`) using `numpy.roll`. This caused over 1.17 million calls to the `roll` function, taking 37.073 seconds of internal execution time.

This directly reflects the core trade-off of the Matrix-Free approach: trading extra CPU calculations for significant memory savings. Although repeatedly calculating the spatial central difference at each CG iteration increases the raw arithmetic cost, it completely removes the need to build and store a massive sparse matrix. This prevents the Out-Of-Memory (OOM) risks common with direct solvers like LU decomposition. Coupled with unconditional stability, this architecture offers high reliability and memory efficiency for large grids.

The subsequent table provides a comprehensive breakdown of the most time-consuming functions within the Matrix-Free implicit solver.

Table 6: cProfile Results for Matrix-Free Implicit Solver (Top Functions)

Function / Module	Calls	Total Time (s)	Per Call (s)	Cum. Time (s)
<code>numeric.py: roll()</code>	1173864	37.073	0.000	38.893
<code>Implicit.py: _apply_laplacian()</code>	293466	34.679	0.000	73.671
<code>iterative.py: cg()</code>	100000	25.204	0.000	108.235
<code>models.py: Grey_Scott()</code>	50000	19.151	0.000	19.151
<code>numpy.ndarray: dot()</code>	293464	17.476	0.000	17.476
<code>Implicit.py: run()</code>	1	11.154	11.154	174.376
<code>numpy.ndarray: flatten()</code>	493466	5.722	0.000	5.722
<code>Implicit.py: step()</code>	50000	5.559	0.000	144.067
<code>Implicit.py: _apply_lhs_matrix()</code>	193466	3.478	0.000	54.580
<code>numpy.ufunc: reduce()</code>	100000	3.167	0.000	3.167
<code>Implicit.py: _apply_rhs_matrix()</code>	100000	1.677	0.000	26.240
<code>builtins: numpy.array</code>	100000	1.455	0.000	1.455
<code>_linalg.py: norm()</code>	293464	1.397	0.000	19.205
<code>numeric.py: normalize_axis_tuple()</code>	1173864	1.104	0.000	1.393
<code>utils.py: make_system()</code>	100000	0.837	0.000	4.080

6 Conclusion

In this project, we explored Turing instability within reaction-diffusion systems and successfully replicated the formation of biological patterns, such as leopard spots and giraffe patches, through numerical simulation.

To rigorously verify our numerical methods, we implemented a dedicated testing suite using the Method of Manufactured Solutions (MMS) and self-convergence tests. This successfully proved that our shared central difference spatial discretization achieves strict $\mathcal{O}(\Delta x^2)$ accuracy, while the explicit and implicit temporal integration schemes perfectly align with their theoretical $\mathcal{O}(\Delta t)$ and $\mathcal{O}(\Delta t^2)$ convergence orders.

Beyond the theoretical validation, our work focused heavily on addressing the performance bottlenecks observed in baseline CPU implementations. To overcome the explicit method’s strict time-step constraints (CFL conditions) and heavy memory bandwidth usage, we introduced a GPU-accelerated architecture using CuPy. By leveraging custom CUDA kernels and zero-copy memory management, we achieved massive data-level parallelism, reducing the runtime of 50,000 iteration steps to under one second.

To resolve the massive memory overhead caused by direct solvers in the implicit CN scheme, we developed a Matrix-Free architecture paired with a Conjugate Gradient (CG) iterative solver. This approach calculates the spatial Laplacian dynamically, completely skipping the assembly of $N^2 \times N^2$ sparse matrices. It keeps the spatial memory complexity strictly bounded to $\mathcal{O}(N^2)$ and avoids the slow $\mathcal{O}(N^3)$ processing time of direct solvers.

In conclusion, this report demonstrates that deep algorithmic refactoring and hardware-accelerated optimizations can effectively break through the physical limitations of baseline numerical methods. We not only validated the complex mathematics behind biological pattern formation but also delivered a highly robust, mathematically proven, and efficient framework for solving large-scale partial differential equations.

References

- [1] A. M. Turing, “The chemical basis of morphogenesis,” *Philosophical Transactions of the Royal Society of London. Series B, Biological Sciences* **237**, 37–72 (1952).
- [2] S. Kondo and T. Miura, “Reaction-Diffusion Model as a Framework for Understanding Biological Pattern Formation,” *Science* **329**, 1616–1620 (2010).
- [3] J. E. Pearson, “Complex Patterns in a Simple System,” *Science* **261**, 189–192 (1993).
- [4] R. T. Liu, S. S. Liaw, and P. K. Maini, “Two-stage Turing model for generating pigment patterns on the leopard and the jaguar,” *Physical Review E* **74**, 011914 (2006). doi: 10.1103/PhysRevE.74.011914.
- [5] A. Kolmogorov, I. Petrovsky, and N. Piscounov, “Étude de l’équation de la diffusion avec croissance de la quantité de matière et son application à un problème biologique,” *Moscow Univ. Bull. Math.* **1**, 1–25 (1937).
- [6] H. Meinhardt, *Models of biological pattern formation* (Academic Press, London, 1982).
- [7] J. D. Murray, “How the leopard gets its spots,” *Sci. Am.* **258**, 80–87 (1988). doi: 10.1038/scientificamerican0388-80.
- [8] J. Schnakenberg, “Simple chemical reaction systems with limit cycle behaviour,” *Journal of Theoretical Biology* **81**, 389–400 (1979). doi: 10.1016/0022-5193(79)90042-0.
- [9] Y. Nishiura and D. Ueyama, “Spatio-temporal chaos for the Gray–Scott model,” *Physica D: Nonlinear Phenomena* **150**(3-4), 137–162 (2001). doi: 10.1016/S0167-2789(00)00214-1.
- [10] H. Meinhardt, “Turing’s theory of morphogenesis of 1952 and the subsequent discovery of the crucial role of local self-enhancement and long-range inhibition,” *J. R. Soc. Interface* **6**, 407–416 (2012). doi: 10.1098/rsfs.2011.0097.
- [11] A. D. Economou *et al.*, “Periodic stripe formation by a Turing mechanism operating at growth zones in the mammalian palate,” *Nat. Genet.* **44**, 348–351 (2012). doi: 10.1038/ng.1090.
- [12] S. Nishihara and T. Ohira, “The Bifurcation Growth Rate for the Robust Pattern Formation in the Reaction-Diffusion System on the Growing Domain,” *J. Phys. Soc. Jpn.* **94**(9), 094804 (2025).